



CompSci 230

Object Oriented Software Development

Expressions & Selection Structure



Review Quizzes

- ▶ Which of the following converts a String (e.g. "12") to an int?
 - A. `int num = (String) value;`
 - B. `int num = Integer.parseInt(value);` ✓
 - C. `int num = value;`
 - ▶ The statement: `System.out.printf("%6.2f", 597.7231);` displays:
 - A. 597.72
 - B. 597.723
 - C. 000597.72 ✓
 - D. None of the others
 - ▶ Which of the following statement displays Hello World?
 - A. `System.out.printf( "%s %s", "Hello", "World" );` ✓
 - B. `System.out.printf( "%2s", "Hello " "World" );`
 - C. `System.out.printf( "%s%s", "Hello,World" );`
 - D. `System.out.printf( "s% s%", "Hello", "World" );`
-



Today's Agenda

► Topics:

► Expressions

- Arithmetic Expressions
- Relational Expressions
- Logical Expressions

► Control Structures

- sequence structure
- selection structure
 - The if , if-else, nested-if statements , the ?: operator, the Switch statement

► Reading:

► Java how to program Late objects version (D & D)

- Chapter 2 & Chapter 3

► The Java Tutorials

- Lesson: Language Basics: Operators, Expressions, Statements, and Blocks
 - <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html>
- Lesson: Language Basics: Control Flow Statements
 - <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html>

► Welcome to Java for Python Programmers

- <http://interactivepython.org/courselib/static/java4python/index.html>



2. Expressions

- ▶ An expression is code that can be evaluated to give a single value:
 - ▶ Combination of variables, constants, operators, and parentheses
 - ▶ Examples of expressions

35

age

3 * number + 56

age / value + (number - age)

- ▶ Arithmetic Expressions
- ▶ Relational Expressions
- ▶ Logical Expressions



3. Arithmetic Expressions

The Arithmetic Operators

- ▶ Arithmetic expression

- ▶ Combine variables and constants with arithmetic operators and parentheses

Operator	Description
+	Additive operator (also used for String concatenation)
-	Subtraction operator
*	Multiplication operator
/	Division operator
%	Modulus / Remainder operator

- ▶ Note:

- ▶ The arithmetic operators are binary operators because they each operate on two operands.
 - ▶ Integer division yields an integer quotient. `10 / 4` ← Result is 2
 - ▶ Any fractional part in integer division is simply truncated (i.e., discarded)—no rounding occurs.
 - ▶ The remainder operator, %, yields the remainder after division.



3. Arithmetic Expressions

Mixing Types

- When a Java operator is applied to operands of different types, Java does a **widening conversion** automatically, known as a promotion.

<code>2.2 * 2</code>	evaluates to 4.4
<code>1.0 / 2</code>	evaluates to 0.5
<code>double x = 2;</code>	assigns 2.0 to x

- Conversions are done on one operator at a time in the order the operators are evaluated.

`3 / 2 * 3.0 + 8 / 3`

5.0

`2.0 * 4 / 5 + 6 / 4.0`

3.1

7.16666





3.Arithmetic Expressions

The Unary Operators

Operator	Description
+	Unary plus operator; indicates positive value (numbers are positive without this, however)
-	Unary minus operator; negates an expression
++	Increment operator; increments a value by 1
--	Decrement operator; decrements a value by 1
!	Logical complement operator; inverts the value of a boolean

► Example:

```
int a = 0;  
a++;  
System.out.println(a) ;  
  
int b = 4;  
b--;  
System.out.println(b) ;
```

1
3



3.Arithmetic Expressions

Post-increment Vs Pre-increment

- ▶ **Post Increment : $i++$**
 - ▶ Increment Value of Variable After Assigning
- ▶ **Pre Increment : $++i$**
 - ▶ First Increment Value and then Assign Value.

```
int i = 0, j = -1;
j = i++;
System.out.println(j);
```

0

```
int p = 0, q = -1;
q = ++p;
System.out.println(q);
```

1

- ▶ **Post-decrement: $i--$, Pre-decrement: $--i$**



Exercise 1

- ▶ A) What is the output of the following code fragment?

```
String phrase = "123";  
System.out.println(phrase*2);
```



- ▶ B) What is the output of the following program?

```
public class L03Ex01{  
    public static void main(String[] args) {  
        int num1=100, num2=1, num3;  
        num3 = num2++ + ++num1;  
        System.out.print(" num1=" + num1);  
        System.out.print(" num2=" + num2);  
        System.out.print(" num3=" + num3);  
    }  
}
```



3.Arithmetic Expressions

The Assignment Operators

Operator	Description	
----------	-------------	--

+=	Add AND assignment operator	C += A is equivalent to C = C + A
----	-----------------------------	-----------------------------------

-=	Subtract AND assignment operator	C -= A is equivalent to C = C - A
----	----------------------------------	-----------------------------------

*=	Multiply AND assignment operator	C *= A is equivalent to C = C * A
----	----------------------------------	-----------------------------------

/=	Divide AND assignment operator	C /= A is equivalent to C = C / A
----	--------------------------------	-----------------------------------

%=	Modulus AND assignment operator	C %= A is equivalent to C = C % A
----	---------------------------------	-----------------------------------

► Example:

```
int x = 1;  
x += 2;  
System.out.println(x);  
  
int y = 8;  
y /= 2;  
System.out.println(y);
```

```
3  
4
```



3.Arithmetic Expressions

Multiple assignments

► Multiple assignments

► Embed **assignment expressions** within **assignment expressions**

- Example: `s = 5 + (t = 4)`
- Evaluates to 9 while `t` is assigned 4

```
int s = 1, t = 2;  
s = 5 + (t = 4);  
System.out.println(s);  
System.out.println(t);
```

9
4



3.Arithmetic Expressions

The + operator

- ▶ The + operator can be used in two ways.
 - ▶ as a concatenation operator
 - ▶ as an addition operator
- ▶ If either side of the + operator is a string, the result will be a string.
- ▶ Example:

```
int value = 5;  
System.out.println("Hello " + "World");  
System.out.println("The value is: " + 5);  
System.out.println("The value is: " + value);  
System.out.println("The value is: " + '\n' + 5);
```

```
Hello World  
The value is: 5  
The value is: 5  
The value is:  
5
```



3.Arithmetic Expressions

Math

- ▶ The `java.lang.Math` class contains various methods for performing basic numeric operations such as the logarithm, cube root, and trigonometric functions

Method Description

`Math.abs()` It will return the Absolute value of the given value.

`Math.max()` It returns the Largest of two values.

`Math.min()` It is used to return the Smallest of two values.

`Math.round()` It is used to round of the decimal numbers to the nearest value.

`Math.sqrt()` It is used to return the square root of a number.

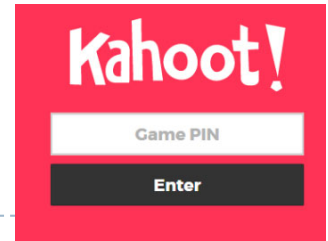
`Math.pow()` It returns the value of first argument raised to the power to second argument.

...



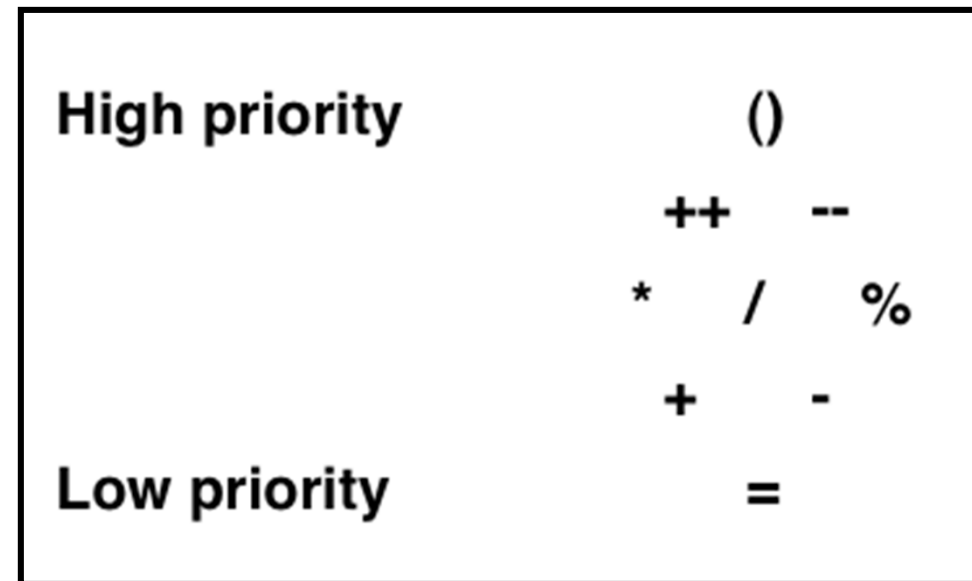
3.Arithmetic Expressions

Order of precedence



- ▶ The evaluation of expressions follows these two rules:
 - ▶ highest precedence operators are always evaluated first
 - ▶ if operators have the same precedence, then evaluation is from left to right

Operator type	Operator	Associates
grouping	<i>(expression)</i>	Left to right
unary	++, --, +, -	Right to left
cast	<i>(type)</i>	Right to left
multiplicative	*, /, %	Left to right
additive	+, -	Left to right
assignment	=, +=, -=, *=, /=, %=	Right to left





4. Relational Expressions

▶ Relational expressions

- ▶ Combine variables and constants with relational, or comparison, and equality operators and parentheses
 - ▶ Relational or comparison operators: $<$, $<=$, $>=$, $>$
 - ▶ Equality operators: $==$, $!=$
- ▶ Evaluate to true or false

```
int value = 5;
```

```
System.out.println(value > 5);
```

false

▶ Note: Common programming Error

- ▶ Confusing the equality operator `'=='` with the assignment operator `'='`

```
int a = 5;
```

```
System.out.println(a=5);
```

```
System.out.println(a==5);
```

5

true



Exercise 2 & 3

- ▶ What is the output of the following program?

```
int a, b;
double c;
System.out.println(7 + 1 + "4 * 2" + 3 + 1);
System.out.println("5" + (7 + 1) + 3 * 2 + 1);
a = 5;
b = a / 3 + 1;
c = a / b;
System.out.println(a + ", " + b + ", " + c);
```

```
int val1 = 50;
int val2 = 53;
System.out.println("1. " + (val1 != val2));
System.out.println("2. " + (val1 >= val2 - 3));
System.out.println("3. " + (67 % 2 == 1));
```




5. Logical Expressions

- ▶ Logical expressions
 - ▶ Combine variables and constants of arithmetic types, relational expressions with logical operators
 - ▶ Logical And: `&&`
 - ▶ Logical Or: `||`
 - ▶ Logical unary NOT: `!`
 - ▶ Evaluate to true or false
 - ▶ Note: Comparison operators can not be chained

`1 < value < 100`



```
value > 1 && value < 100
value >= 1 || value == 5
```



5.Logical Expressions

Order of precedence

- ▶ Logical operators in the order of precedence

	Operator	Meaning	Example
high	!	Not	! (a == b)
	&&	And	(a == b) && (c ==d)
low		Or	(a == b) (c ==d)

- ▶ Example: expression `a && !b || c` would be evaluated as `(a && (!b)) || c`
- ▶ Parentheses can be used to change the order of evaluation
 - ▶ Example: `a && (!b || c)`



5.Logical Expressions

More Examples

is the value greater than 10 and less than 100

value > 10 && value < 100

is the value greater than 10 or is the value equal to 1

value > 10 || value == 1

is the value not greater than 10

!(value > 10)

(value <= 10)

is the value not greater 10 and not equal to 1

either

value <= 10 && value != 1

or

!(value > 10 || value == 1)



5.Logical Expressions

Short-circuit evaluation

	Meaning	Short circuit?
&&	and	yes
&	and	no
	or	yes
	or	no

▶ Short-circuit evaluation

- ▶ Stops as soon as the value of expression is determined

▶ Truth table:

A	B	And Operator	Or operator
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

- ▶ The OR operator results in true when A is true, no matter what B is.
- ▶ Similarly, the AND operator results in false when A is false, no matter what B is.

`System.out.println(a>0 || value>a/0);`

true

true

Skip the second part as the result is always TRUE

`System.out.println(a>10 || value>a/0);`

false

Must evaluate the second part -> generate an error

Exception in thread "main" java.lang.ArithmeticException: / by zero



5.Logical Expressions Common Errors

Mathematics shortcuts do not work in Java.



```
if (10 < b < 20) {  
    doSomething();  
}
```

Not valid in
Java



```
if (10 < b && b < 20) {
```

Remember that the equality test uses double equals ==



```
if (a = b) {  
    doSomething();  
}
```

Tries to do
assignment



```
if (a == b) {
```

Summary

Operator Precedence

- Operations with higher order of precedence are applied first



	Operation	Symbol
Higher order of precedence ↓ low	Grouping operators	()
	Unary operators	+, -, !
	Multiplicative arithmetic	*, /, %
	Additive arithmetic	+, -
	Relational ordering	<, >, <=, >=
	Relational equality	==, !=
	Logical and	&&
	Logical or	
	Assignment	=, +=, -=, *=, /=, %=

Exercise 4



- ▶ What is the output of the following program?

```
int value = 12, x = 0, y = 3;  
boolean result;
```



High Precedence!

```
result = value>10 || value<=5 && value!=12;  
System.out.println("1. " + result);
```

```
result = (value>10 || value<=5) && value!=12;  
System.out.println("2. " + result);
```

```
System.out.print("3: ");  
System.out.println(x == 0 || (y / x) == 3);
```

```
System.out.print("4: ");  
System.out.println(x != 0 && (y / x) == 3);
```

Exercise 5

- Write a program that averages the rain fall for three months, April, May, and June. Declare and initialize a variable to the rain fall for each month. Compute the average, and print the result to 2 decimal places.

```
//import

public class L03Ex05 {
    public static void main(String[] args) {
        int april, may, june;
        double average;
        //create a scanner object

        //calculate the average
        average = (april + may + june) / 3 ;
        System.out.printf("Average rainfall: %.2f\n", average);
    }
}
```

```
Rainfall for April: 12
Rainfall for May: 14
Rainfall for June: 8
Average rainfall: 11.33
```

What is wrong here?
What should you
use instead of "3"?

```
Rainfall for April: 12
Rainfall for May: 14
Rainfall for June: 8
Average rainfall: 11.00
```





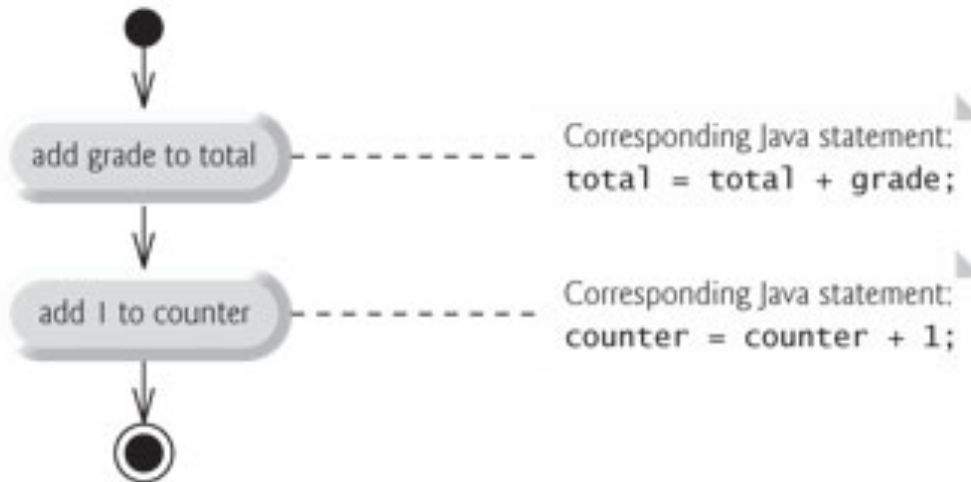
6. Control Structures

- ▶ **Sequential execution**
 - ▶ Statements in a program execute one after the other in the **order** in which they are written.
- ▶ **Transfer of control**
 - ▶ Various Java statements, enable you to specify that the next statement to execute is not necessarily the next one in sequence.
- ▶ **All programs can be written in terms of only three control structures—**
 - ▶ the sequence structure
 - ▶ the selection structure
 - ▶ the repetition structure

6. Sequence Structure

► Sequence structure

- Built into Java.
- Unless directed otherwise, the computer executes Java statements one after the other in the order in which they're written.



```
System.out.println("Hello");  
System.out.println("World");
```

```
Hello  
World
```



7. Selection Structure

- ▶ Three types of selection statements.
- ▶ **if statement:**
 - ▶ Performs an action, if a condition is true; skips it, if false.
 - ▶ Single-selection statement—selects or ignores a single action (or group of actions).
- ▶ **if...else statement:**
 - ▶ Performs an action if a condition is true and performs a different action if the condition is false.
 - ▶ Double-selection statement—selects between two different actions (or groups of actions).
- ▶ **switch statement**
 - ▶ Performs one of several actions, based on the value of an expression.
 - ▶ Multiple-selection statement—selects among many different actions (or groups of actions).



7. Selection Structure

The if statement

- ▶ if Single-Selection Statement

- ▶ Python and Java are very similar

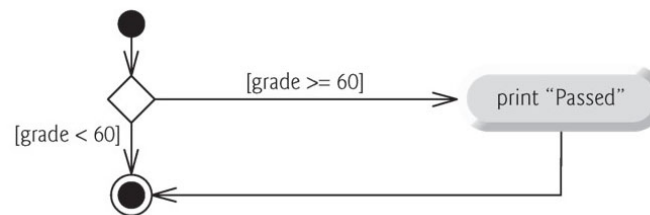
Python

```
if condition:  
    statement1  
    ...
```

```
if (condition) {  
    statement1;  
    ...  
}
```

Java

- ▶ The statements in the consequent and each alternative are marked by **curly braces** ({}). When there is just one statement, the braces may be omitted.
 - ▶ Indentation: Optional, but recommended
 - ▶ Each Boolean expression is enclosed in **parentheses**.
 - ▶ Example:
 - ▶ If the condition is false, the Print statement is ignored, and the next pseudocode statement in order is performed.





7.Selection Structure

The if-else statement

Python

```
if condition:
    statement1
    ...
else:
    statement2
    ...
```

► if...else Double-Selection Statement

- specify an action to perform when the condition is true and another action when the condition is false.

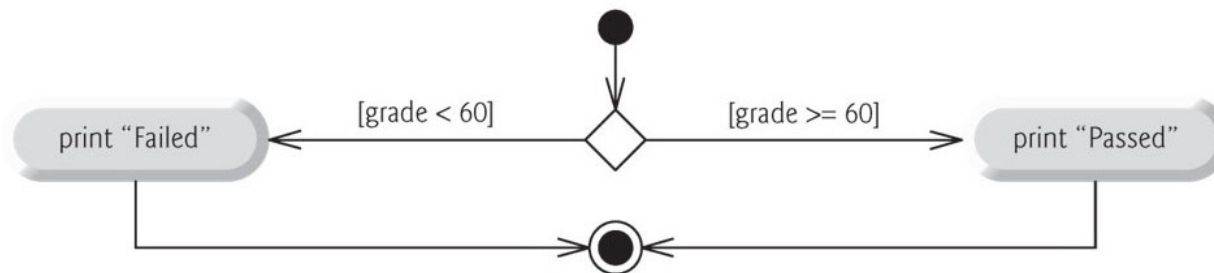
Only one statement
in its body! Don't
have to use {}

```
if (condition)
    statement1;
else
    statement2;
```

```
if (condition) {
    statement1;
    ...
} else {
    statement2;
    ...
}
```

several statements in the body,
must enclose the statements in
braces {}.
i.e. to form a block

► Example:





7.Selection Structure

Nested if-else statements

- ▶ Nested if...else statements:
 - ▶ placing if...else statements inside other if...else statements

```
if (grade <= 60) {  
    System.out.println('D');  
} else if (grade < 75) {  
    System.out.println('C');  
} else if (grade < 90) {  
    System.out.println('B');  
} else  
    System.out.println('A');
```

In a nested if-else statements, ensure that you test for all possible cases!



7. Selection Structure

if-else Double-Selection Statement

- ▶ The Java compiler always associates an else with the **immediately preceding if** unless told to do otherwise by the placement of braces
- ▶ The following code is not what it appears:

```
if (x > 5)
    if (y > 5)
        System.out.println("x and y are > 5");
else
    System.out.println("x is <= 5");
```



- ▶ **Beware!** This nested if...else statement does not execute as it appears. The compiler actually interprets the statement as

```
if (x > 5)
    if (y > 5)
        System.out.println("x and y are > 5");
else
    System.out.println("x is <= 5");
```

- ▶ To force the nested if...else statement to execute as it was originally intended, we must write it as follows:

```
if (x > 5) {
    if (y > 5)
        System.out.println("x and y are > 5");
} else
    System.out.println("x is <= 5");
```

{ and }



7. Selection Structure

The ? : Operator

- ▶ This operator consists of three operands and is used to evaluate Boolean expressions.
- ▶ The goal of the operator is to decide which value should be assigned to the variable.
- ▶ It is a ternary operator (takes three operands)
- ▶ The operator is written as:

```
variable x = (expression) ? value if true : value if false
```

- ▶ Operand to the left of the ? is a boolean expression
 - evaluates to a boolean value (true or false)
 - ▶ Second operand (between the ? and :) is the **value** if the boolean expression is true
 - ▶ Third operand (to the right of the :) is the **value** if the boolean expression evaluates to false.
 - ▶ It can be used in place of an if...else statement.
-



7. Selection Structure

The ? : Operator - Examples

Passed

▶ Example 1:

```
int grade=60;  
System.out.println(grade >= 60 ? "Passed" : "Failed");
```

- ▶ Evaluates to the string "Passed" if the boolean expression `grade >= 60` is true and to the string "Failed" if it is false.

▶ Example 2:

Value of b is : 30
Value of b is : 20

```
int a, b;  
a = 10;  
b = (a == 1) ? 20: 30;  
System.out.println( "Value of b is : " + b );  
  
b = (a == 10) ? 20: 30;  
System.out.println( "Value of b is : " + b );
```



7. Selection Structure

The switch Statement

- ▶ A **switch** statement allows a variable to be tested for **equality** against a list of values.
- ▶ Each value is called a **case**, and the variable being switched on is checked for each case.
 - ▶ Variables: **integers**, convertible integers (**byte**, **short**, **char**) and **strings**
 - ▶ Value: must be the **same** data type, a **constant** or a **literal**.
 - ▶ When the variable being switched on is equal to a case, the statements following that case will execute until a **break** statement is reached.
 - ▶ Not every case needs to contain a break. If no break appears, the flow of control will **fall through** to subsequent cases until a break is reached.
 - ▶ A switch statement can have an optional **default case**, which must appear at the end of the switch.
 - The default case can be used for performing a task when none of the cases is true. No break is needed in the default case.



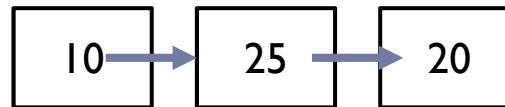
7.Selection Structure Example

- ▶ What would be the value of x after the following statements were executed?

```
int x = 10;  
switch (x)  
{  
    case 10:  
        x += 15;  
    case 12:  
        x -= 5;  
        break;  
    default:  
        x *= 3;  
}
```



If no break appears, the flow of control will **fall through from case 10 to case 12**



x=20



Multiple-Selection Statement

► Note:

- If a **match** occurs, the program executes that case's statements.
- switch does **not** provide a mechanism for testing **ranges of values**—every value must be listed in a separate case label.
- switch differs from other control statements in that it does **not** require **braces** around multiple statements in a case.
- Without **break**, the statements for a matching case and subsequent cases execute until a break or the end of the switch is encountered. This is called “**falling through**.”
- If no match occurs between the controlling expression's value and a case label, the **default case executes**.
- If no match occurs and there is no default case, program control simply continues with the first statement after the switch.

